

NoRBERT: Transfer Learning for Requirements Classification

Summary: The paper introduces NoRBERT, a BERT fine tuning approach for automatically classifying natural language software requirements, aimed at fixing the common issue that traditional feature based ML models lose accuracy on unseen projects due to changes in wording and style. Using the PROMISE NFR dataset and a relabeled version, the authors evaluate several tasks including functional vs non functional classification, predicting major non functional subclasses, and distinguishing functional versus quality aspects, showing NoRBERT matches or outperforms prior work while requiring no manual feature engineering or preprocessing and maintaining strong performance in cross project evaluations. They also label the functional aspect requirements with concern tags Function, Data, and Behavior and release this dataset, then show NoRBERT can detect Function and Behavior well but struggles more on Data mainly because the class is underrepresented and recall is low.

Three positive point:

1. **Strong generalization to unseen projects:** Authors don't just report random cross-validation. They explicitly test project-aware splits and show NoRBERT stays accurate even when the test requirements come from projects the model never saw before.
2. **Minimal manual work and clean pipeline:** NoRBERT removes the need for handcrafted linguistic features and heavy text preprocessing used in older approaches, which makes the method simpler to reproduce and easier to apply to new datasets.
3. **New useful labeled dataset and extra task:** Beyond benchmarking on PROMISE and the relabeled dataset, they add and release a new annotation layer for functional requirements (Function, Data, Behavior) and demonstrate it can be learned reasonably well, opening the door to more fine grained requirements analysis.

Three Negative points:

1. **Small and imbalanced datasets limit what the results prove**
PROMISE is relatively small and many NFR subclasses have very few examples, so performance can look strong on common classes but drop on rare ones, and the model has limited chance to learn minority categories well.
2. **Lower performance on harder or underrepresented labels (especially "Data")**
In their new Function Data Behavior labeling, the model is noticeably weaker on the Data concern, mainly because Data examples are fewer and the classifier misses many true Data cases (low recall).
3. **Less interpretability**
It is harder to explain exactly why a requirement was assigned a label, which can

matter in requirements engineering practice.

EXAM:

What is the context of this work?

This paper is in the area of requirements engineering and natural language processing. It studies how to automatically **classify software requirements** written in plain language, especially into **functional and non functional types**. Earlier methods often worked on known datasets, but their performance became worse on unseen projects because wording and writing style change from project to project. The authors use **BERT, a transfer learning language model**, because it can learn general language patterns from large text data and then be adapted to a smaller task. Their system is called NoRBERT, and they test it on the PROMISE NFR dataset, a relabeled version of that dataset, and a new functional concern dataset they created.

What problem is this work trying to address?

The main problem is that existing requirement classification methods do not generalize well to new and unseen projects. Many older approaches **depend too much on project specific wording, sentence style, or manual preprocessing**, so they are less useful in real practice. The paper tries to build a classifier that keeps good performance on both seen and unseen projects, while also improving classification of non functional subclasses and functional concerns.

Methodology

What is the study design?

The authors **fine-tune pre-trained BERT models**, both base and large versions, to classify requirements, and they call the approach NoRBERT. They **test it on four tasks: functional vs non functional classification, non functional subclass classification, classification on the relabeled functional and quality aspect dataset, and a new task for classifying functional requirements into function, data, and behavior**. They use established datasets, especially the PROMISE NFR dataset and the relabeled version by Dalpiaz et al., and they also manually label functional requirements to create a new dataset. To evaluate performance, they **use precision, recall, and F1 score, with settings such as 10 fold cross validation, project specific folds, and leave one project out (10 fold cross validation splits all data into 10 parts and tests each part once, project specific folds train on some projects and test on different projects, and leave one project out trains on all projects except one and tests on that left out project.)** They also compare NoRBERT directly with earlier published methods to see whether transfer learning improves results.

Why have the authors chosen this particular methodology?

They chose **transfer learning** (Transfer learning is a machine learning technique where a model developed for one task is reused as the starting point for a second, related task) **with BERT** because requirements engineering usually has only a small amount of labeled data, and **BERT can work well even with limited task-specific data**. They also wanted a method that needs less manual feature engineering and preprocessing than older approaches. Most importantly, **they wanted to test whether a transfer learning model can generalize better to unseen projects**, which is the main weakness of previous work.

What is the methodology attempting to evaluate?

The methodology evaluates whether NoBERT **improves requirement classification performance compared with earlier methods**. It also evaluates whether the model can still perform well **when tested on new projects** it did not see during training. In addition, it checks whether transfer learning can classify not only broad categories, but also fine grained subclasses such as non functional subclasses and functional concerns like function, data, and behavior.

Results

What is the key takeaway/lessons learned of this paper? (3 sentences)

The main takeaway is that **NoBERT works very well** and usually matches or beats previous methods, **especially on harder settings with unseen projects**. It performs similarly on basic functional vs non functional classification, **but it is stronger on non functional subclasses**, the relabeled dataset, and the new functional concern task, while avoiding manual preprocessing. **The biggest lesson is that transfer learning makes requirements classification more practical**, because it improves generalization and **can work with limited labeled data**.

Automated Handling of Anaphoric Ambiguity in Requirements: A Multi-solution Study

This paper studies how to **automatically handle pronoun ambiguity** in natural language requirements by comparing six different NLP and machine learning approaches on about 1,350 industrial requirements. **It finds that supervised machine learning is best for detecting ambiguous pronouns (about 60% precision with 100% recall)**. At the same

time, **a fine tuned SpanBERT model is best for resolving pronouns** to the right antecedent (about 98% success). Hence, the authors recommend a hybrid workflow that uses ML to flag risky cases and SpanBERT to suggest likely references when ambiguity is low.

Three positive points:

1. **Comprehensive comparison that gives actionable guidance:** The paper does a true multi solution study with six alternative approaches (language model, supervised ML, and off the shelf coreference tools), so it can be learn which technology fits which task instead of seeing only one proposed method.
1. **Evaluation on realistic industrial requirements with careful annotation:** It uses a large and diverse industrial dataset (DAMIR: 22 specifications across 8 domains, 1,251 requirements, 737 pronoun occurrences) labeled by two independent non author annotators, and reports moderate agreement ($\kappa = 0.54$), which supports credibility and generalizability.
2. **Clear performance win and practical recommendation:** The results are strong and easy to apply: best ambiguity detection achieves $\approx 60\%$ precision and 100% recall, and best resolution achieves $\approx 98\%$ success rate, leading to a simple recommended hybrid workflow (ML for detection, SpanBERT for resolution).

Three negative points:

1. **Annotation agreement is only moderate, and ambiguity labels partly come from disagreement:** they report Fleiss' $\kappa = 0.54$, which they call "moderate agreement," and when annotators disagreed for a pronoun, the authors treat that pronoun as ambiguous and label all its triples inconclusive. This is reasonable, but it also means the ground truth for "ambiguous" is closely tied to annotator disagreement rather than an independent gold standard definition.
2. **Precision for ambiguity detection is still around 60%, so it can create many false alarms:** the best ambiguity detector (MLensemble) achieves 100% recall but only $\sim 60\%$ precision, which implies a substantial number of false positives that analysts would need to filter manually.
3. **Generalizability is not fully settled and needs more validation in practice** Even though DAMIR spans eight domains, the authors explicitly note external validity threats and say that more documents and user studies are needed to more conclusively evaluate usefulness.

EXAM

What is the context of this work?

This paper is about requirements engineering and natural language processing. It

studies anaphoric ambiguity, which happens when a **pronoun like it or they can refer to more than one possible thing in a requirement**. This kind of ambiguity is common in natural language requirements and can cause different readers to understand the same requirement differently. The authors focus on **automating both ambiguity detection and anaphora resolution**, meaning finding whether a pronoun is ambiguous and, when possible, identifying what it refers to. They compare several modern and traditional AI and NLP methods on industrial requirements to see which works best.

What problem is this work trying to address?

The paper tries to solve the problem that pronouns in requirements can be unclear and misleading, which may cause misunderstandings in software projects. Previous work in requirements engineering used older NLP and ML methods and was often tested on small datasets or only one domain. **The authors want a more accurate and practical automated solution for both detecting ambiguous pronouns and resolving clear anaphora in real industrial requirements.**

What is the study design?

The study is a **comparative experimental study** where the authors build and compare **six different automated solutions** for the same problem, which is handling pronoun ambiguity in requirements. These six solutions are grouped into **three families: two SpanBERT based solutions (LLM), three supervised machine learning solutions, and one NLP coreference solution** built from existing tools. **Before testing, they preprocess each requirement to detect pronouns, collect the nearby text as context, and generate all possible noun phrases that the pronoun might refer to.** Then they **train, fine tune, and tune** the models using datasets including **CoNLL2011, ReqEval**, and their own industrial dataset **DAMIR**, which contains requirements from several domains. Finally, they compare all solutions on **three things**: how well they detect ambiguity, how well they resolve the pronoun, and how much time they take to run.

Why have the authors chosen this particular methodology?

They chose this methodology because there are many possible NLP and ML technologies, and they wanted to **compare them side by side** instead of assuming one is best. They also wanted to check whether recent language models like **BERT and SpanBERT** are actually better than traditional methods for this requirements task. A multi solution experiment helps them find not only the best tool, but also useful lessons for other requirements engineering automation problems.

What is the methodology attempting to evaluate?

The methodology evaluates which solution is best at **detecting anaphoric ambiguity** in requirements. It also evaluates which solution is best at **resolving pronouns**, meaning identifying the correct antecedent when possible. In addition, it checks the **execution time** of each solution to see whether the methods are practical to use.

What is the key takeaway/lessons learned of this paper?

The main takeaway is that **different tasks need different best solutions**. For **ambiguity detection**, the best result comes from **supervised machine learning**, especially the ensemble model, with about **60 percent precision and 100 percent recall**. For **anaphora resolution**, the best result comes from **SpanBERTRE (LLM)**, which reaches about **98 percent success rate**, so the authors recommend a **hybrid approach** that combines ML for detection and SpanBERT for resolution.